

Гаптическая система

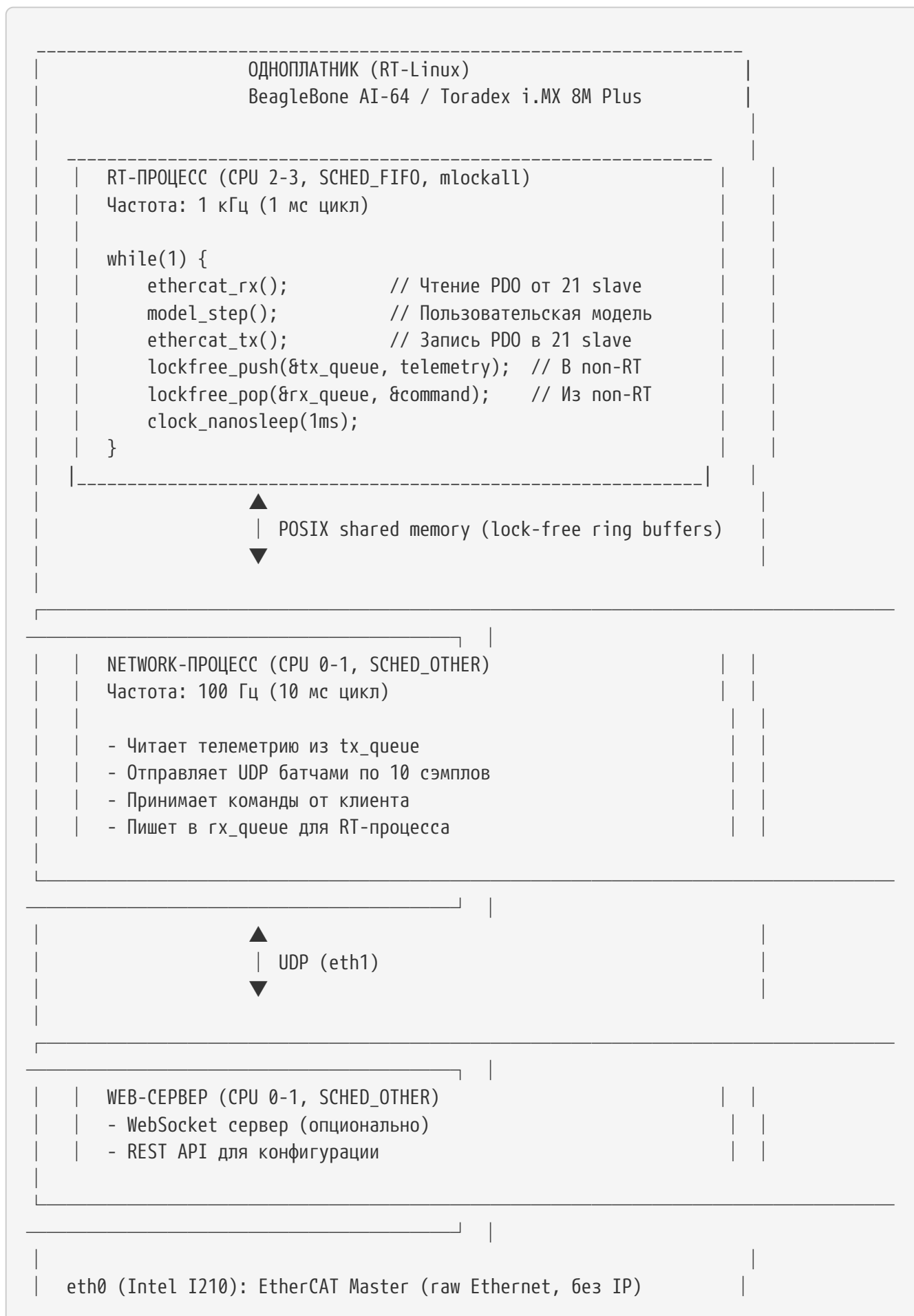
Владислав Богданов

2026-07-02

Оглавление

1. Общая архитектура	1
2. Выбор железа	3
2.1. Одноплатник: BeagleBone AI-64 (рекомендуется)	3
2.2. EtherCAT Master: IgH EtherCAT Master (open-source)	3
3. Настройка RT-Linux	4
3.1. Ядро: PREEMPT_RT (встроено в mainline с 5.15+)	4
4. Архитектура EtherCAT	4
4.1. Конфигурация шины	4
4.2. Расчет времени цикла (1 кГц)	5
4.3. Режимы работы приводов	5
5. Структура данных	5
5.1. RT-процесс → Network-процесс (телеметрия)	5
5.2. Network-процесс → RT-процесс (команды)	6
6. Интеграция с MATLAB/Simulink	6
6.1. Вариант А: Simulink модель генерирует C код для RT-процесса	6
6.2. Вариант Б: MATLAB External Mode для online-тюнинга	7
7. Защита RT-детерминизма	7
7.1. Двухуровневая архитектура	8
7.2. Автоматическая валидация	8
8. Web-интерфейс	9
8.1. Архитектура	9
8.2. Пример кода (Python на Host PC)	10
9. Итоговые характеристики	11
10. Преимущества выбранной архитектуры	11
11. Следующие шаги	11
12. Итог	12

1. Общая архитектура



eth1 (USB-GbE): LAN → Host PC (UDP, WebSocket)

EtherCAT (100 Мбит/с, 1 кГц)

ETHERCAT SHINE (21 slave)

КАНАЛ 1 (3 slave)

- АЦП датчика силы (16-24 бит, 1 кГц)
- Драйвер сервопривода (CST mode, 1 кГц)
- Энкодер положения (24+ бит, 1 кГц)

КАНАЛ 2 (3 slave)

- АЦП датчика силы
- Драйвер сервопривода
- Энкодер положения

...

КАНАЛ 7 (3 slave)

- АЦП датчика силы
- Драйвер сервопривода
- Энкодер положения

Итого: 7 каналов × 3 slave = 21 slave устройство

UDP (100 Hz, binary)

HOST PC (Windows/Linux)

	Python script / C++ приложение	
	- Прием UDP телеметрии	
	- Отправка команд	
	- Визуализация (uPlot, PyQtGraph)	
	MATLAB/Simulink (опционально)	
	- External Mode для online-тюнинга	
	- Мониторинг в реальном времени	

2. Выбор железа

2.1. Одноплатник: BeagleBone AI-64 (рекомендуется)

Преимущества: * TI TDA4VM: 6x ARM Cortex-A72 @ 2.0 ГГц + 2x PRU + DSP * RAM: 4 ГБ DDR4 (достаточно для Linux + RT) * Ethernet: 2x GbE (один для EtherCAT, второй для LAN) * PRU: можно использовать для EtherCAT Master (hard real-time) * MATLAB Support Package: официальный от MathWorks * Цена: ~\$150-200

Альтернативы: * Toradex Verdin i.MX 8M Plus: промышленный, дороже (\$300+), но с гарантией RT * Raspberry Pi CM4: дешевле (\$100), но менее предсказуемый RT

2.2. EtherCAT Master: IgH EtherCAT Master (open-source)

Преимущества: * Бесплатный, open-source * Работает как kernel module * Поддерживает Distributed Clocks (DC) * Хорошая документация * Используется в промышленности

Альтернатива: Acontis (коммерческий, ~5000 EUR, но с Simulink Blockset)

3. Настройка RT-Linux

3.1. Ядро: PREEMPT_RT (встроено в mainline с 5.15+)

Boot parameters (uEnv.txt):

```
# Изоляция CPU 2-3 для RT-поточков
isolcpus=2,3
nohz_full=2,3
rcu_nocbs=2,3

# Отключение CPU frequency scaling
cpufreq.default_governor=performance

# Отключение watchdog (опционально)
nowatchdog
```

Настройка IRQ affinity:

```
# Привязка IRQ eth0 (EtherCAT) к CPU 2
echo 4 > /proc/irq/<IRQ_ETH0>/smp_affinity

# Привязка IRQ eth1 (LAN) к CPU 0
echo 1 > /proc/irq/<IRQ_ETH1>/smp_affinity
```

Ожидаемый джиттер: 10-50 мкс (при правильной настройке)

4. Архитектура EtherCAT

4.1. Конфигурация шины

```
Master (одноплатник)
|
|─► Slave 1: АЦП канала 1 (TxPDO: 4 байта force)
|─► Slave 2: Драйвер канала 1 (RxPDO: 4 байта torque, TxPDO: 4 байта status)
|─► Slave 3: Энкодер канала 1 (TxPDO: 4 байта position)
|
|─► Slave 4: АЦП канала 2
|─► Slave 5: Драйвер канала 2
|─► Slave 6: Энкодер канала 2
|
... (каналы 3-7)
|
|─► Slave 19: АЦП канала 7
|─► Slave 20: Драйвер канала 7
```

4.2. Расчет времени цикла (1 кГц)

Компонент	Время (мкс)
EtherCAT frame transmission (21 slave × 12 байт PDO)	25
Slave processing (21 × 3 мкс)	63
Distributed Clocks sync	20
Модель управления (7 каналов, импеданс)	500
Margin (джиттер, прерывания)	200
ИТОГО	808 мкс □

Запас: 192 мкс (достаточно для джиттера)

4.3. Режимы работы приводов

- **CST (Cyclic Synchronous Torque):** основной режим для гаптики
 - Master шлет target torque каждый цикл
 - Привод отрабатывает токовый контур
 - Полный контроль над импедансом
- **Distributed Clocks (DC):** обязательны
 - Синхронизация всех slave < 1 мкс
 - Применение команд в строго заданный момент времени

5. Структура данных

5.1. RT-процесс → Network-процесс (телеметрия)

```
typedef struct {
    // Канал 1
    float force_1;           // от АЦП
    float position_1;        // от энкодера
    float torque_1;          // от драйвера (actual)
    uint32_t status_1;       // статус драйвера

    // Канал 2-7 (аналогично)
    float force_2;
    float position_2;
    float torque_2;
    uint32_t status_2;
    // ...
}
```

```

float force_7;
float position_7;
float torque_7;
uint32_t status_7;

uint64_t timestamp;    // время в наносекундах
uint32_t cycle_count;  // счетчик циклов
} telemetry_sample_t;  // ~120 байт

```

5.2. Network-процесс → RT-процесс (команды)

```

typedef struct {
    // Параметры для всех каналов
    float target_position[7]; // целевые позиции
    float stiffness[7];       // виртуальная жесткость
    float damping[7];         // виртуальное демпфирование

    uint32_t mode;             // режим: 0=STOP, 1=RUN, 2=HOME
    uint32_t sequence;         // номер команды
} command_t;                  // ~92 байта

```

6. Интеграция с MATLAB/Simulink

6.1. Вариант А: Simulink модель генерирует С код для RT-процесса

Настройка:

```

% Configuration Parameters (Ctrl+E)
Solver: Fixed-step discrete
Fixed-step size: 0.001 (1 мс = 1 кГц)
Hardware Board: Generic → Unspecified (assume 32-bit Generic)
System target file: ert.tlc (Embedded Real-Time)
Toolchain: ARM GCC (кросс-компиляция)

```

Модель:

Simulink Model (1 кГц)

[EtherCAT Read] → [7 каналов: force, position, torque]

[Command from Network] —► [target_pos, stiffness, damping] |

Канал 1:

[Impedance Control]

$F = K \cdot (x_{\text{target}} - x) - D \cdot v$

└► [torque_cmd_1]

... (каналы 2-7)

[torque_cmd_1..7] —► [EtherCAT Write]

[position_1..7, force_1..7] —► [Telemetry to Network]

Генерация кода:

% Ctrl+B → генерирует model.c, model.h

% Кросс-компиляция: arm-linux-gnueabi-gcc -O2 model.c -o model.o

6.2. Вариант Б: MATLAB External Mode для online-тюнинга

На хост-машине:

% 1. Открываете Simulink модель

% 2. Tools → Run on Target Hardware → Prepare to Run

% 3. Target: Linux on BeagleBone

% 4. Monitor and Tune

% Результат:

% - Модель работает на одноплатнике

% - Можно менять K_p , K_d , stiffness в реальном времени

% - Видите графики в Simulink Scope

% - НЕ НУЖНО перепрошивать

7. Защита RT-детерминизма

7.1. Двухуровневая архитектура

УРОВЕНЬ 1: RT-ЯДРО (ЗАЩИЩЕНО)

- Фиксированный цикл 1 кГц
- EtherCAT Rx/Tx
- Вызов пользовательской модели
- Watchdog: если модель > 800 мкс → emergency stop
- Пользователь НЕ редактирует этот код

Вызов функции

УРОВЕНЬ 2: ПОЛЬЗОВАТЕЛЬСКАЯ МОДЕЛЬ (РЕДАКТИРУЕМАЯ)

```
function torque = user_model(pos, vel, force)
    %#codegen
```

% Разрешено:

- % - Математика: +, -, *, /, sin, cos
- % - Фиксированные массивы
- % - Условные операторы

% Запрещено:

- % - malloc, fprintf, rand
- % - Системные вызовы

```
error = target_pos - pos;
torque = Kp * error - Kd * vel;
```

```
end
```

Валидация перед деплоем:

- Проверка на запрещенные функции
- Профилирование времени выполнения
- Если > 800 мкс → отказ в деплое

7.2. Автоматическая валидация

```
% validate_and_deploy.m
function deploy_model(model_name)
    % 1. Проверка синтаксиса
```

```

validate_model_syntax(model_name);

% 2. Генерация C кода
codegen -config cfg model_name;

% 3. Проверка сгенерированного кода
if grep('malloc|fprintf|rand', 'model.c')
    error('Forbidden functions detected');
end

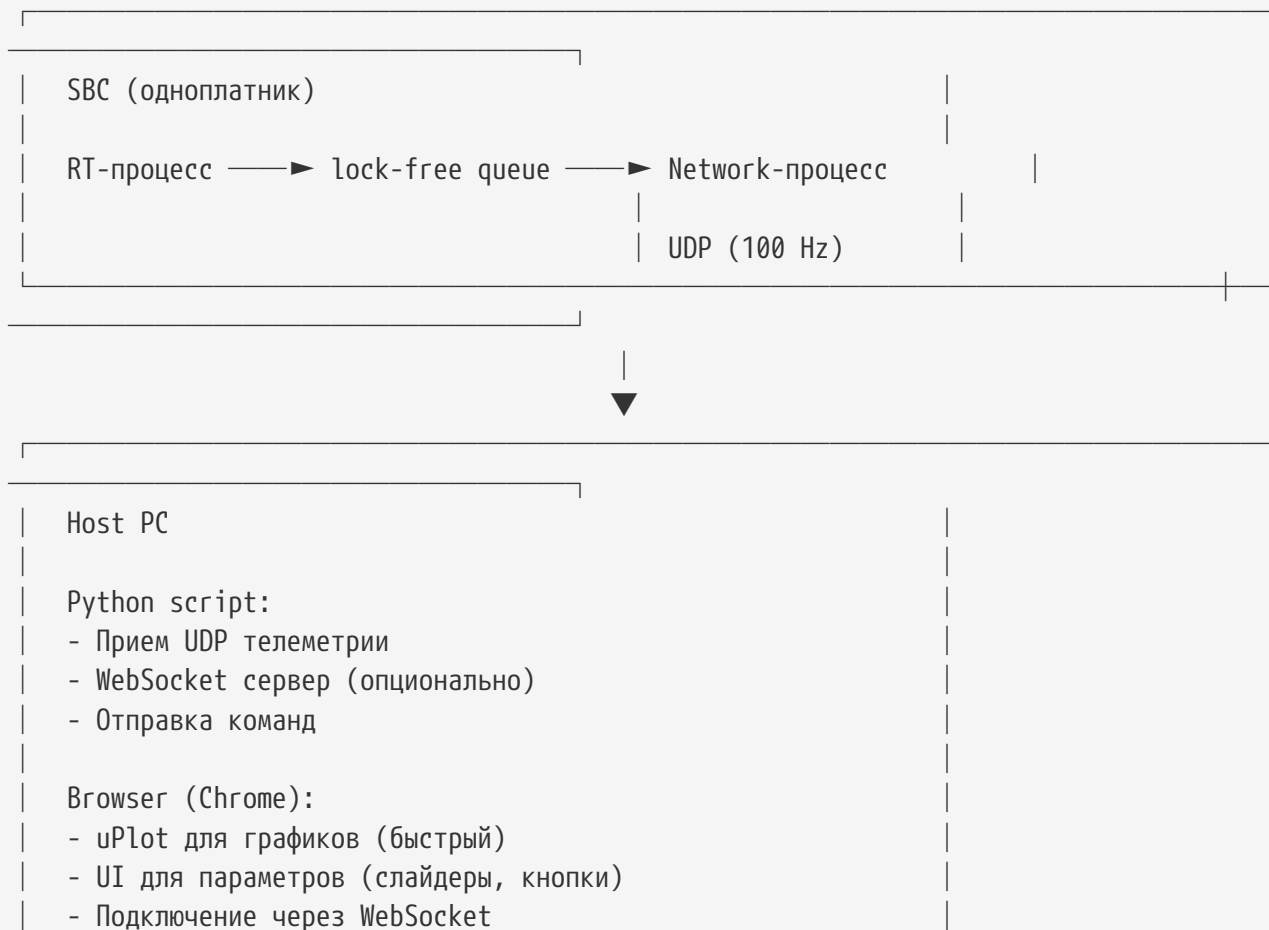
% 4. Профилирование
avg_time = benchmark_model();
if avg_time > 800
    error('Model too slow: %.2f us', avg_time);
end

% 5. Деплой
deploy_to_target();
end

```

8. Web-интерфейс

8.1. Архитектура



8.2. Пример кода (Python на Host PC)

```
#!/usr/bin/env python3
import socket
import struct
import asyncio
import websockets

# UDP прием телеметрии
udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
udp_sock.bind(("0.0.0.0", 9001))

# WebSocket сервер
connected_clients = set()

async def ws_handler(websocket, path):
    connected_clients.add(websocket)
    try:
        async for message in websocket:
            # Прием команд от клиента
            cmd = parse_command(message)
            send_command_to_sbc(cmd)
    finally:
        connected_clients.remove(websocket)

async def udp_to_ws():
    while True:
        data, addr = udp_sock.recvfrom(4096)
        # Парсинг телеметрии
        telemetry = parse_telemetry(data)

        # Отправка всем подключенным клиентам
        if connected_clients:
            message = struct.pack("fff", telemetry.pos, telemetry.vel,
telemetry.force)
            await asyncio.gather(
                *[client.send(message) for client in connected_clients]
            )

async def main():
    await asyncio.gather(
        websockets.serve(ws_handler, "0.0.0.0", 8080),
        udp_to_ws()
    )
```

9. Итоговые характеристики

Параметр	Значение
Одноплатник	BeagleBone AI-64 (TI TDA4VM)
Ядро	Linux 6.x + PREEMPT_RT
Частота RT-цикла	1 кГц (1 мс)
Джиттер	10-50 мкс
EtherCAT Master	IgH EtherCAT Master (open-source)
Количество slave	21 (7 каналов × 3 устройства)
Время цикла EtherCAT	~800 мкс (запас 200 мкс)
Distributed Clocks	Да (синхронизация < 1 мкс)
Режим приводов	CST (Cyclic Synchronous Torque)
Пользовательская модель	Simulink → C code (ert.tlc)
Online-тюнинг	MATLAB External Mode
Web-интерфейс	UDP → Python → WebSocket → Browser
Защита RT	Watchdog, валидация модели, cgroups

10. Преимущества выбранной архитектуры

- **Простота разработки:** один одноплатник, нет необходимости в STM32 и CAN
- **Гибкость:** пользователь может редактировать модель, hot-reload без перепрошивки
- **Масштабируемость:** легко добавить больше каналов (до 50 slave на 1 кГц)
- **Интеграция с MATLAB:** полная поддержка Simulink, External Mode
- **Web-интерфейс:** мониторинг и управление через браузер
- **Защита RT:** двухуровневая архитектура, watchdog, валидация
- **Промышленный стандарт:** EtherCAT, PREEMPT_RT — используются в реальной промышленности

11. Следующие шаги

1. Закупка железа:

- BeagleBone AI-64 (~\$200)
- USB-to-Ethernet адаптер (Intel I210) (~\$30)

с. EtherCAT slave устройства ($7 \times$ АЦП, $7 \times$ драйвер, $7 \times$ энкодер)

2. Настройка RT-Linux:

- a. Установка PREEMPT_RT ядра
- b. Настройка isolcpus, nohz_full
- с. Установка IgH EtherCAT Master

3. Разработка базовой модели:

- a. Simulink модель для 1 канала (тест)
- b. Генерация C кода
- с. Интеграция с EtherCAT

4. Тестирование:

- a. Измерение джиттера (cyclictest)
- b. Проверка времени цикла
- с. Тестирование 7 каналов

5. Разработка web-интерфейса:

- a. UDP сервер на SBC
- b. WebSocket сервер на Host PC
- с. Frontend на uPlot

6. Интеграция с MATLAB:

- a. External Mode для online-тюнинга
- b. Автоматическая валидация модели

12. Итог

Выбранная архитектура — это **промышленное решение** с RT-Linux на одноплатнике, EtherCAT шиной на 21 slave (7 каналов), интеграцией с MATLAB и web-интерфейсом. Она обеспечивает **детерминизм 10-50 мкс, гибкость для пользователя и масштабируемость** для будущих расширений.